



دستورکار جلسه دهم

۱. برنامه ای به زبان C بنویسید که سیگنال **SIGSEGV** (مربوط به خطای **segmentation fault**) را هندل کند و در صورت وقوع سیگنال مربوطه، آدرس حافظه‌ای که باعث خطای **segmentation fault** شده است را نمایش دهد و برنامه را متوقف کند.

حال در ادامه برنامه خود سعی کنید بدون استفاده از تابع **kill** یک سیگنال **SIGSEGV** ایجاد کنید و خروجی را مشاهده کنید.

۲. هدف از این سوال ارسال ساختارمند اطلاعات از طریق حافظه‌ی مشترک در دو پردازنده است. برای این منظور مثال اول پیش گزارش بخش حافظه مشترک را در نظر بگیرید و گام های زیر را انجام دهید. (زمان پیشنهادی: ۸۰ دقیقه)

گام اول:

- ابتدا در فایل هدر برنامه (**protocol.h**)، **struct** زیر را تعریف کنید:

```
struct player {  
    char player_name[NAME_LEN];  
    int score;  
};
```

هدف ما در این گام این است که در یک برنامه، اطلاعات بازیکنان را در حافظه مشترک نوشته و در یک برنامه دیگر آنها را نشان دهیم. بدین منظور دو برنامه زیر را بنویسید :

- برنامه **score_updater** :

این فرآیند اطلاعات بازیکنان و امتیازات آنها را به جدول امتیازات اضافه می‌کند:

♦ اطلاعات بازیکن را در جدول ثبت کنید. (یک آرایه به تعداد **NUM** از این ساختار بسازید. **NUM** را در

فایل **protocol.h** تعریف کنید و به صورت پیش فرض آن را برابر ۷ قرار دهید)

♦ برای امتیاز بازیکنان از اعداد تصادفی بین ۱ تا ۱۰۰ استفاده کنید.

♦ نام بازیکنان را به دلخواه مقدار دهی کنید

۲. برنامه `score_viewer`:

- این فرآیند بعد از ثبت اطلاعات توسط برنامه `score_update` اجرا میشود. اطلاعات جدول امتیازات را از حافظه مشترک خوانده و آن را نمایش می‌دهد.

گام دوم:

در این گام میخواهیم فرایند ثبت و خواندن اطلاعات را به کمک سیگنال هندل کنیم. بدین منظور هر دو برنامه را بدین شکل تغییر دهیم:

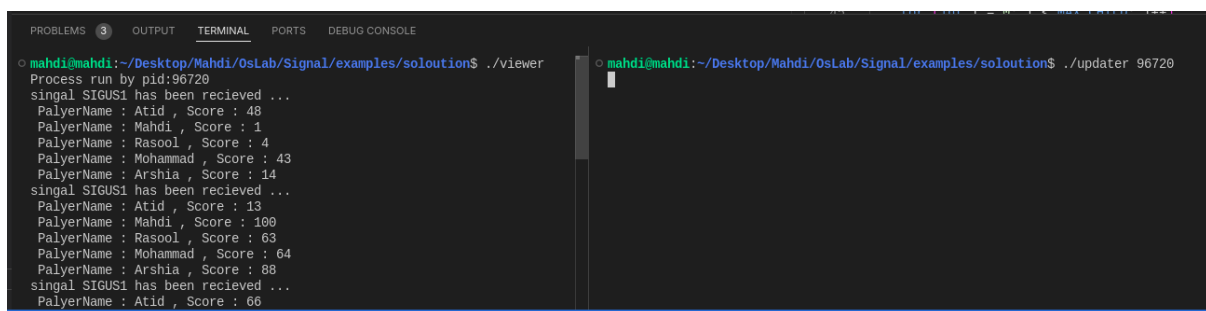
• `score_updater`:

برنامه `score_updater` را طوری بازنویسی کنید که به صوت دوره ای اطلاعات حافظه مشترک را به روز کند. بدین منظور هر ۳ ثانیه یکبار امتیاز بازیکنان را دوباره به صورت رندوم مقداردهی کرده و جایگزین اطلاعات قبلی کند (نیازی به تغییر نام بازیکنان نیست). سپس بعد از نوشتن اطلاعات در حافظه مشترک سیگنال `SIGHUSR1` را به برنامه `score_viewer` ارسال کند.

• `score_viewer`:

برنامه `score_viewer` را طوری بازنویسی کنید که همواره منتظر دریافت سیگنال `SIGHUSR1` باشد و به محض دریافت این سیگنال، اطلاعات را از حافظه مشترک بخواند و آن را به نمایش درآورد. و همچنین بعد از نمایش اطلاعات، دوباره منتظر دریافت سیگنال بماند.

راهنمایی: برای این که راحت تر سیگنال را ارسال کنید، در برنامه `score_viewer` به محض اجرا، `pid` اش را چاپ کنید و عددی که نشان میدهد را به آرگومان های تابع `main` در برنامه `score_updater` ارسال کنید تا به کمک آن سیگنال `SIGHUSR1` را ارسال کند. (مشابه تصویر زیر)



```
PROBLEMS 3 OUTPUT TERMINAL PORTS DEBUG CONSOLE
mahdi@mahdi:~/Desktop/Mahdi/OsLab/Signal/examples/solution$ ./viewer
Process run by pid:96720
singal SIGHUSR1 has been recieved ...
PalyerName : Atid , Score : 48
PalyerName : Mahdi , Score : 1
PalyerName : Rasool , Score : 4
PalyerName : Mohammad , Score : 43
PalyerName : Arshia , Score : 14
singal SIGHUSR1 has been recieved ...
PalyerName : Atid , Score : 13
PalyerName : Mahdi , Score : 100
PalyerName : Rasool , Score : 63
PalyerName : Mohammad , Score : 64
PalyerName : Arshia , Score : 88
singal SIGHUSR1 has been recieved ...
PalyerName : Atid , Score : 66

mahdi@mahdi:~/Desktop/Mahdi/OsLab/Signal/examples/solution$ ./updater 96720
```

گام سوم :

در این گام می‌خواهیم کاری که قبلاً فرآیند `score_updater` به تنهایی انجام میداد را این بار به کمک فرآیند های فرزندش انجام دهد.

بدین منظور، در برنامه `score_updater` به تعداد `MAX_CHILD` فرزند ایجاد کنید و برای هر کدام یک `id` منحصر به فرد ارسال کنید. حال می‌خواهیم عمل به روزرسانی اطلاعات بازیکنان را بین فرزندان تقسیم کنیم. شما باید در اینجا یک روش تقسیم کار منصفانه ارائه دهید. یعنی بین همه ی پردازنده‌ها تقریباً یکسان کار تقسیم شود. همچنین در مواردی که تعداد عناصر آرایه (کارها) بر تعداد فرزندان بخش پذیر نبود باز برنامه باید صحیح کار کند. به همین دلیل پیشنهاد می شود هر فرزند به این شکل عمل نماید (فرض کنید `MAX_CHILD` ۵ است):

فرزند صفر: اندیس های ۰ و ۵ و ۱۰ و ۱۵ و

فرزند یک: اندیس های ۱ و ۶ و ۱۱ و ۱۶ و

فرزند دوم: اندیس های ۲ و ۷ و ۱۲ و ۱۷ و ...

فرزند سوم: اندیس های ۳ و ۸ و ۱۳ و ۱۸ و ...

فرزند چهارم: اندیس های ۴ و ۹ و ۱۴ و ۱۹ و

به همین دلیل می توانید از حلقه ی زیر استفاده کنید:

```
for(int i = child_id; i < ARRAY_SIZE; i += MAX_CHILD)
```

والد باید منتظر پایان کار تمام فرزندان بماند و در نهایت مانند گام دوم اطلاعات را در حافظه مشترک نوشته و بعد از آن سیگنال `SIGUSR1` بفرستد (مانند گام قبل این فرآیند باید هر ۳ ثانیه یکبار توسط برنامه `score_updater` انجام شود و جدول دائماً توسط فرآیند `score_viewer` به روز شود).